# UNIT – II: DEEP LEARNING

## PART A: SHORT ANSWER QUESTIONS (2 MARKS EACH)

Easy English answers with generated sketch diagrams

### Q1. Define Multi-Layer Perceptron (MLP). How does it overcome the fundamental limitations of a single-layer perceptron?

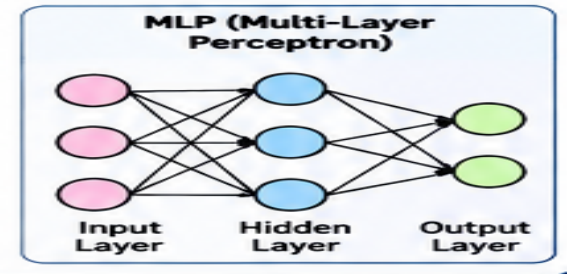
A **Multi-Layer Perceptron (MLP)** is an artificial neural network that contains an input layer, one or more hidden layers and an output layer.

It overcomes the limitations of a single-layer perceptron because:

1. It has multiple layers to learn complex patterns.
2. It can solve non-linear problems.
3. It uses activation functions to produce better results.

**Example:** Recognizing handwritten digits.

**Generated illustration:**



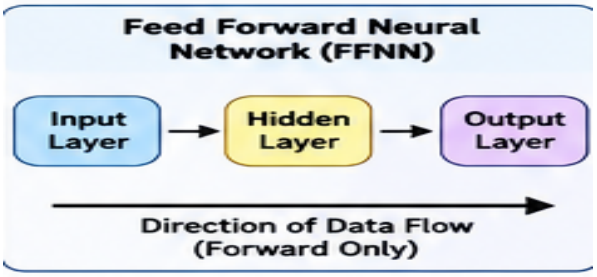

### Q2. What is a Feed Forward Neural Network (FFNN)? State the direction of data flow.

A **Feed Forward Neural Network (FFNN)** is a neural network in which data moves only in one direction.

Data moves from the input layer to the hidden layer and then to the output layer.

1. Data moves forward only.
2. There are no backward connections.
3. It is used for prediction and classification.

**Generated illustration:**




### Q3. List any three activation functions used in hidden layers (Sigmoid, Tanh, ReLU) with their mathematical expressions and output ranges.

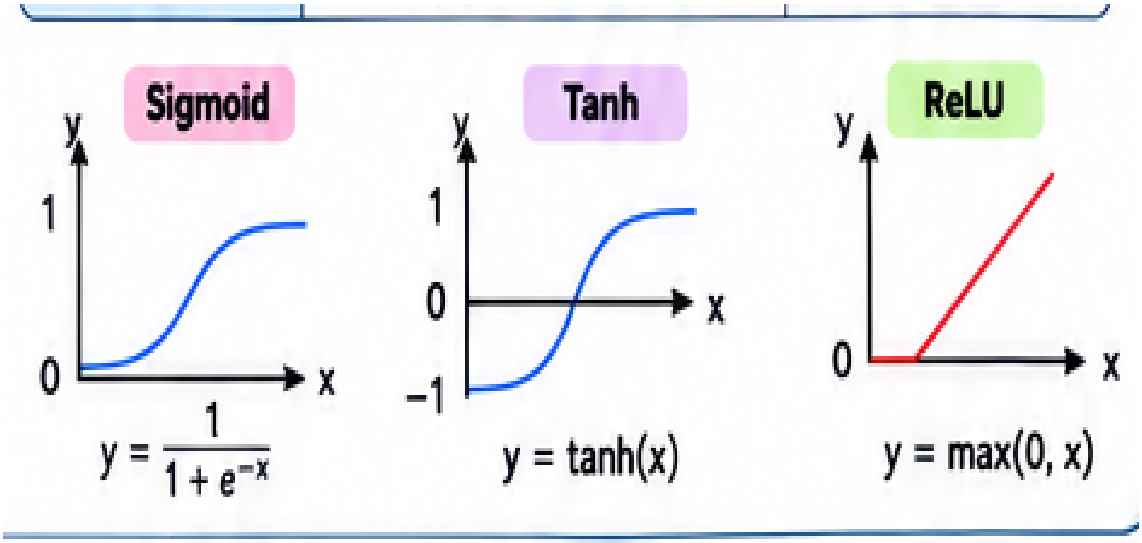
An **Activation Function** decides whether a neuron should pass its output to the next layer.

**1. Sigmoid:** Formula: f(x) = 1 / (1 + e■■). Output range: 0 to 1.

**2. Tanh:** Formula: f(x) = tanh(x). Output range: –1 to +1.

**3. ReLU:** Formula: f(x) = max(0, x). Output range: 0 to infinity.

**Generated illustration:**




## Q4. What are the three variants of Gradient Descent (Batch GD, Stochastic GD, Mini-batch GD)? Compare their convergence speed.

**Gradient Descent** is a method used to reduce the error of a neural network by updating its weights.

**1. Batch Gradient Descent:** Uses the complete training dataset for one update. Stable but usually slow for large datasets.

**2. Stochastic Gradient Descent (SGD):** Uses only one training example for each update. Updates quickly but may move irregularly.

**3. Mini-Batch Gradient Descent:** Uses a small group of training examples for each update. Provides a balance between speed and stability.

| Type | Data used per update | Convergence / update speed |
|---|---|---|
| Batch GD | Entire dataset | Slower updates |
| SGD | One example | Fast updates; may be noisy |
| Mini-batch GD | Small group | Balanced |

**Simple sketch:**

```
Batch GD:      All data     -> One update
SGD:           One example -> One update
Mini-batch GD: Small group  -> One update
```

## Q5. What are the common Loss / Cost functions used in Deep Learning (MSE, Binary Cross-Entropy, Categorical Cross-Entropy) and their applications?

A **Loss Function** measures how far the predicted answer is from the correct answer.

**1. Mean Squared Error (MSE):** Measures the average squared difference between actual and predicted values. Used for regression problems.
**Formula:** MSE = (1/n) Σ(y – ■)²

**2. Binary Cross-Entropy:** Used when there are only two classes. Example: Spam or Not Spam.

**3. Categorical Cross-Entropy:** Used when there are more than two classes. Example: Cat, Dog or Bird.

**Generated illustration:**

and their applications?

**Answer:**

| Loss / Cost Function | Mathematical Expression | Application |
|---|---|---|
| Mean Squared Error (MSE) | $MSE = \frac{1}{n}\sum_{i=1}^{n}(y_i - \hat{y}_i)^2$ | Regression (e.g., house price) |
| Binary Cross-Entropy (BCE) | $L = -[y \log(\hat{y}) + (1-y)\log(1-\hat{y})]$ | Binary classification (e.g., Spam / Not Spam) |
| Categorical Cross-Entropy (CEE) | $L = -\sum_{i=1}^{C} y_i \log(\hat{y}_i)$ | Multi-class classification (e.g., Cat / Dog / Bird) |

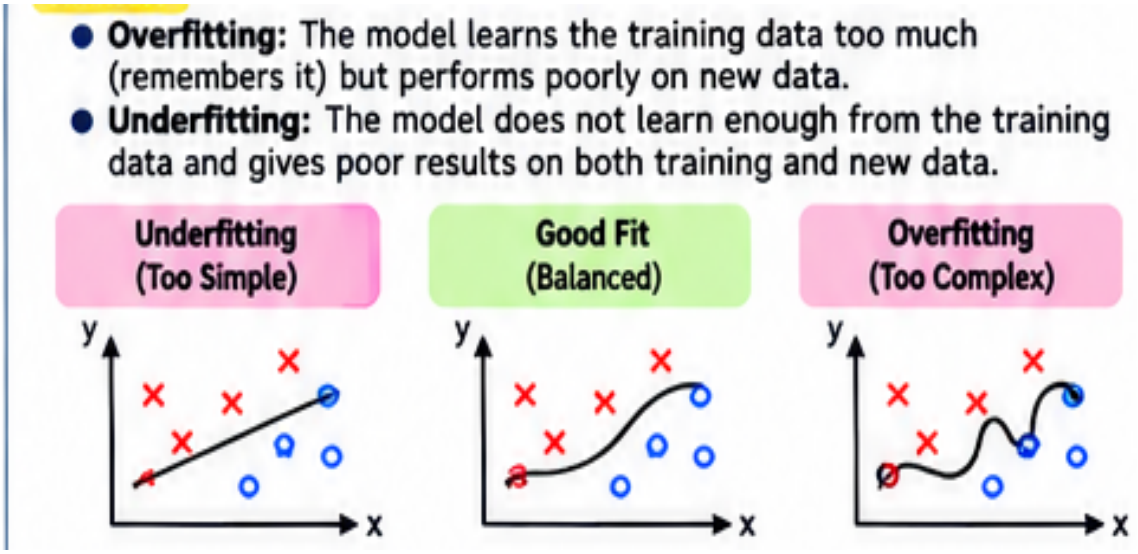
## Q6. Define Overfitting and Underfitting in neural network training.

**1. Overfitting:** Happens when a model learns the training data too much but performs poorly on new data. It remembers training examples and gives poor results on unseen data.

**2. Underfitting:** Happens when a model does not learn enough from the training data. It cannot understand important patterns and gives poor results on both training and new data.

**Good fit:** Learns the useful pattern and works well on new data.

**Generated illustration:**



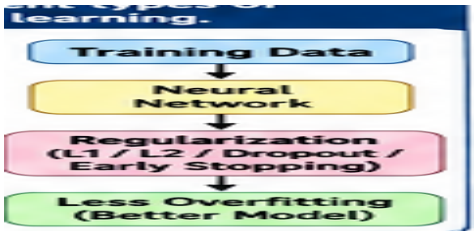

## Q7. Define Regularization and list different types of regularization methods used in deep learning.

**Regularization** is a technique used to prevent overfitting in a neural network. It helps the model perform better on new data.

**Types of Regularization:**

1. **L1 Regularization:** Adds the absolute values of weights to the loss.
2. **L2 Regularization:** Adds the squared values of weights to the loss.
3. **Dropout:** Randomly switches off some neurons during training.
4. **Early Stopping:** Stops training when performance on validation data starts getting worse.

**Generated illustration:**




## Q8. Define Empirical Risk Minimization (ERM) and state its mathematical formulation.

**Empirical Risk Minimization (ERM)** is a method used to find the best model by reducing the average error on training data.

In simple words, it tries to make the model's predictions as close as possible to the correct answers.

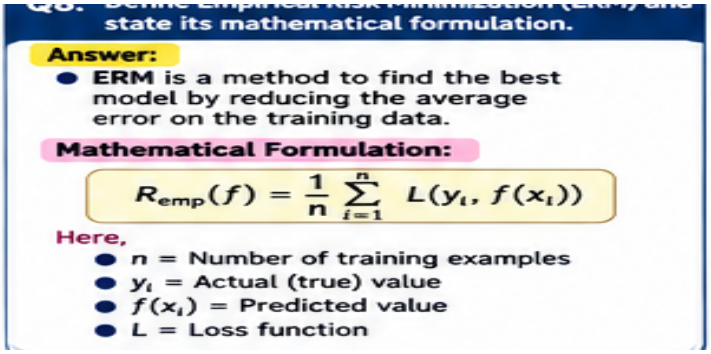
**Mathematical Formula:**

$R_{emp}(f) = (1/n) \Sigma_{i=1}^{n} L(y_i, f(x_i))$

**Where:**

n = Number of training examples.
y■ = Actual answer.
f(x■) = Predicted answer.
L = Loss function.

**Generated illustration:**




## Q9. Explain the Vanishing and Exploding Gradient Problem in deep neural networks.

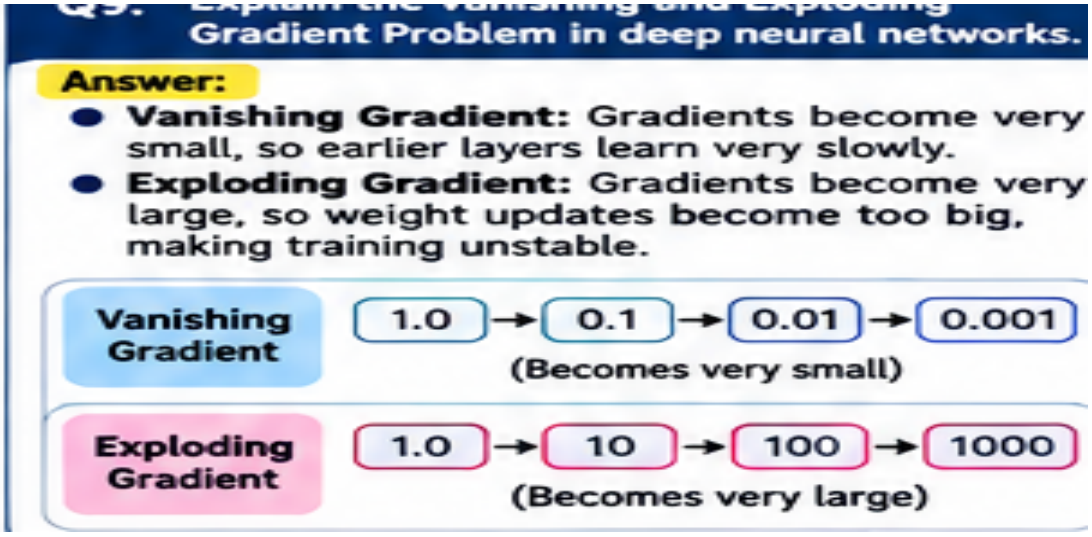
Gradients are values used by a neural network to update its weights during training.

**1. Vanishing Gradient:** Gradients become very small. Earlier layers learn very slowly, so training becomes difficult.

**2. Exploding Gradient:** Gradients become very large. Weight updates become too big, so training becomes unstable.

**Conclusion:** Both problems make deep neural network training difficult.

**Generated illustration:**



## Q10. What is Greedy Layer-Wise Pre-training and why was it historically essential for deep architectures?

**Greedy Layer-Wise Pre-training** is a method of training a deep neural network one layer at a time. Each layer is trained separately before training the complete network.

**Steps:**
1. Train the first layer.
2. Train the second layer using the output of the first layer.
3. Continue the same process for the remaining layers.
4. Finally, train the complete network together using fine-tuning.

**Why was it important?**
It helped deep networks learn better, provided a good starting point for training, and helped reduce training difficulties in older deep networks.

**Generated illustration:**

